

**Московский государственный технический
Университет им. Н.Э. Баумана**

**Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и управления»**

Курс «Парадигмы и конструкции языков программирования»
Отчет по лабораторной работе №3-4
«Функциональные возможности языка Python.

Выполнил:
студент группы ИУ5-31Б
Ростинин Марк

Проверил:
Гапанюк Ю. Е.

2025 г.

Описание задания:

Задача 1 (файл `field.py`)

Необходимо реализовать генератор `field`. Генератор `field` последовательно выдает значения ключей словаря.

В качестве первого аргумента генератор принимает список словарей, дальше через `*args` генератор принимает неограниченное количество аргументов.

Если передан один аргумент, генератор последовательно выдает только значения полей, если значение поля равно `None`, то элемент пропускается.

Если передано несколько аргументов, то последовательно выдаются словари, содержащие данные элементы. Если поле равно `None`, то оно пропускается. Если все поля содержат значения `None`, то пропускается элемент целиком.

Задача 2 (файл `gen_random.py`)

Необходимо реализовать генератор `gen_random(количество, минимум, максимум)`, который последовательно выдает заданное количество случайных чисел в заданном диапазоне от минимума до максимума, включая границы диапазона.

Задача 3 (файл `unique.py`)

Необходимо реализовать итератор `Unique(данные)`, который принимает на вход массив или генератор и итерируется по элементам, пропуская дубликаты.

Конструктор итератора также принимает на вход именованный `bool`-параметр `ignore_case`, в зависимости от значения которого будут считаться одинаковыми строки в разном регистре. По умолчанию этот параметр равен `False`.

При реализации необходимо использовать конструкцию `**kwargs`.

Итератор должен поддерживать работу как со списками, так и с генераторами.

Итератор не должен модифицировать возвращаемые значения.

Задача 4 (файл `sort.py`)

Дан массив 1, содержащий положительные и отрицательные числа. Необходимо **одной строкой кода** вывести на экран массив 2, которые содержит значения массива 1, отсортированные по модулю в порядке убывания. Сортировку необходимо осуществлять с помощью функции `sorted`.

Необходимо решить задачу двумя способами:

1. С использованием `lambda`-функции.
2. Без использования `lambda`-функции.

Задача 5 (файл `print_result.py`)

Необходимо реализовать декоратор `print_result`, который выводит на экран результат выполнения функции.

Декоратор должен принимать на вход функцию, вызывать её, печатать в консоль имя функции и результат выполнения, после чего возвращать результат выполнения.

Если функция вернула список (`list`), то значения элементов списка должны выводиться в столбик.

Если функция вернула словарь (`dict`), то ключи и значения должны выводиться в столбик через знак равенства.

Задача 6 (файл `cm_timer.py`)

Необходимо написать контекстные менеджеры `cm_timer_1` и `cm_timer_2`, которые считают время работы блока кода и выводят его на экран.

После завершения блока кода в консоль должно вывестись `time: 5.5` (реальное время может несколько отличаться).

`cm_timer_1` и `cm_timer_2` реализуют одинаковую функциональность, но должны быть реализованы двумя различными способами (на основе класса и с использованием библиотеки `contextlib`).

Задача 7 (файл `process_data.py`)

В предыдущих задачах были написаны все требуемые инструменты для работы с данными. Применим их на реальном примере.

В файле [`data\_light.json`](#) содержится фрагмент списка вакансий.

Структура данных представляет собой список словарей с множеством полей: название работы, место, уровень зарплаты и т.д.

Необходимо реализовать 4 функции - `f1`, `f2`, `f3`, `f4`. Каждая функция вызывается, принимая на вход результат работы предыдущей. За счет декоратора `@print_result` печатается результат, а контекстный менеджер `cm_timer_1` выводит время работы цепочки функций.

Предполагается, что функции `f1`, `f2`, `f3` будут реализованы в одну строку. В реализации функции `f4` может быть до 3 строк.

Функция `f1` должна вывести отсортированный список профессий без повторений (строки в разном регистре считать равными). Сортировка должна игнорировать регистр. Используйте наработки из предыдущих задач.

Функция `f2` должна фильтровать входной массив и возвращать только те элементы, которые начинаются со слова “программист”. Для фильтрации используйте функцию `filter`.

Функция `f3` должна модифицировать каждый элемент массива, добавив строку “с опытом Python” (все программисты должны быть знакомы с Python).

Функция `f4` должна сгенерировать для каждой специальности зарплату от 100 000 до 200 000 рублей и присоединить её к названию специальности.

Листинг кода:

Файл field.py:

```
def field(items, *args):
    assert len(args) > 0
    for item in items:
        if len(args) == 1:
            val = item.get(args[0])
            if val is not None:
                yield val
        else:
            res = {a: item.get(a) for a in args if item.get(a) is not None}
            if res:
                yield res

if __name__ == '__main__':
    goods = [
        {'title': 'Ковер', 'price': 2000, 'color': 'green'},
        {'title': 'Диван для отдыха', 'color': 'black'}
    ]
    print(list(field(goods, 'title')))
    print(list(field(goods, 'title', 'price')))
```

файл gen_random.py:

```
import random

def gen_random(num_count, begin, end):
    for _ in range(num_count):
        yield random.randint(begin, end)

if __name__ == '__main__':
    print(list(gen_random(5, 1, 3)))
```

Файл unique.py:

```
class Unique(object):
    def __init__(self, items, **kwargs):
        self.ignore_case = kwargs.get('ignore_case', False)
        self.items = iter(items)
        self.used_elements = set()

    def __next__(self):
        while True:
            item = next(self.items)
            val = item.lower() if self.ignore_case and isinstance(item, str)
        else item

            if val not in self.used_elements:
                self.used_elements.add(val)
                return item

    def __iter__(self):
        return self
```

```
if __name__ == '__main__':
    data = ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']
    print(list(Unique(data)))
    print(list(Unique(data, ignore_case=True)))
```

Файл sort.py:

```
data = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]

if __name__ == '__main__':
    result = sorted(data, key=abs, reverse=True)
    print(result)

    result_with_lambda = sorted(data, key=lambda x: abs(x), reverse=True)
    print(result_with_lambda)
```

Файл print_result.py:

```
def print_result(func):
    def wrapper(*args, **kwargs):
        print(func.__name__)
        result = func(*args, **kwargs)
        if isinstance(result, list):
            print('\n'.join(map(str, result)))
        elif isinstance(result, dict):
            for key, val in result.items():
                print(f'{key} = {val}')
        else:
            print(result)
        return result

    return wrapper

@print_result
def test_1(): return 1

@print_result
def test_2(): return 'iu5'

@print_result
def test_3(): return {'a': 1, 'b': 2}

@print_result
def test_4(): return [1, 2]

if __name__ == '__main__':
    test_1()
    test_2()
    test_3()
    test_4()
```

Файл cm_timer.py:

```

import time
from contextlib import contextmanager

class cm_timer_1:
    def __enter__(self):
        self.start_time = time.perf_counter()

    def __exit__(self, exp_type, exp_value, traceback):
        print(f'time: {time.perf_counter() - self.start_time}')

@contextmanager
def cm_timer_2():
    start_time = time.perf_counter()
    yield
    print(f'time: {time.perf_counter() - start_time}')

if __name__ == '__main__':
    with cm_timer_1():
        time.sleep(1)
    with cm_timer_2():
        time.sleep(1)

```

Файл process_data.py:

```

import json
import sys
from lab_python_fp.field import field
from lab_python_fp.gen_random import gen_random
from lab_python_fp.unique import Unique
from lab_python_fp.print_result import print_result
from lab_python_fp.cm_timer import cm_timer_1

path = sys.argv[1] if len(sys.argv) > 1 else 'data_light.json'

with open(path, encoding='utf-8') as f:
    data = json.load(f)

@print_result
def f1(arg):
    return sorted(list(Unique(field(arg, 'job-name'), ignore_case=True)),
key=str.lower)

@print_result
def f2(arg):
    return list(filter(lambda x: x.lower().startswith('программист'), arg))

@print_result
def f3(arg):
    return list(map(lambda x: x + " с опытом Python", arg))

@print_result
def f4(arg):
    return [f'{job}, зарплата {salary} руб.' for job, salary in zip(arg,
gen_random(len(arg), 100000, 200000))]

```

```
if __name__ == '__main__':  
    with cm_timer_1():  
        f4(f3(f2(f1(data))))
```

Примеры работы программы:

```
field x  
:  
C:\Users\Mark\AppData\Local\Programs\Python\Python313\python.exe C:\U  
['Ковер', 'Диван для отдыха']  
[{'title': 'Ковер', 'price': 2000}, {'title': 'Диван для отдыха'}]
```

```
gen_random x  
:  
C:\Users\Mark\AppData\Loc  
[1, 3, 3, 1, 1]
```

```
unique x  
:  
C:\Users\Mark\AppData\L  
['a', 'A', 'b', 'B']  
['a', 'b']
```

```
sort x  
:  
C:\Users\Mark\AppData\Local\Programs\Python\  
[123, 100, -100, -30, 30, 4, -4, 1, -1, 0]  
[123, 100, -100, -30, 30, 4, -4, 1, -1, 0]
```

```
n print_result x  
:  
b = 2  
test_4  
1  
2
```

```
cm_timer x
C:\Users\Mark\AppData\Local\
time: 1.0008510000043316
time: 1.0007658999966225
```

```
process_data x
Программист / Senior Developer
Программист 1C
Программист C#
Программист C++
Программист C++/C#/Java
Программист/ Junior Developer
Программист/ технический специалист
Программист-разработчик информационных систем
f3
Программист с опытом Python
Программист / Senior Developer с опытом Python
Программист 1C с опытом Python
Программист C# с опытом Python
Программист C++ с опытом Python
Программист C++/C#/Java с опытом Python
Программист/ Junior Developer с опытом Python
Программист/ технический специалист с опытом Python
Программист-разработчик информационных систем с опытом Python
f4
Программист с опытом Python, зарплата 138180 руб.
Программист / Senior Developer с опытом Python, зарплата 106916 руб.
Программист 1C с опытом Python, зарплата 178820 руб.
Программист C# с опытом Python, зарплата 147409 руб.
Программист C++ с опытом Python, зарплата 148404 руб.
Программист C++/C#/Java с опытом Python, зарплата 191754 руб.
Программист/ Junior Developer с опытом Python, зарплата 189649 руб.
Программист/ технический специалист с опытом Python, зарплата 166749 руб.
Программист-разработчик информационных систем с опытом Python, зарплата 109011 руб.
time: 0.023469799998565577
```